МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Инженерный бизнес и менеджмент”
Кафедра “Промышленная логистика (Бизнес-информатика)”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №1

Выполнил:
Бухвалова В. А.

Преподаватель:

Правдина А. Д.

Бурмистрова М. В.

Задание

Разработать программу для решения биквадратного уравнения.

1. Программа должна быть разработана в виде консольного приложения на языке Python.
2. Программа осуществляет ввод с клавиатуры коэффициентов A, B, C, вычисляет дискриминант и ДЕЙСТВИТЕЛЬНЫЕ корни уравнения (в зависимости от дискриминанта).
3. Коэффициенты A, B, C могут быть заданы в виде параметров командной строки (вариант задания параметров приведен в конце файла с примером кода). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. Описание работы с параметрами командной строки.
4. Если коэффициент A, B, C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.
5. Дополнительное задание 1 (*). Разработайте две программы на языке Python - одну с применением процедурной парадигмы, а другую с применением объектно-ориентированной парадигмы.
6. Дополнительное задание 2 (*). Разработайте две программы - одну на языке Python, а другую на любом другом языке программирования.

Текст программы

Консольное приложение:

Python:

```python
while True:
    try:
        a = float(input("Введите A: "))
        break
    except ValueError:
        print("Это не число, введите число")

while True:
    try:
        b = float(input("Введите B: "))
        break
    except ValueError:
        print("Это не число, введите число")
```

```
while True:
    try:
        c = float(input("Введите C: "))
        break
    except ValueError:
        print("Это не число, введите число")

if a == 0:
    if b == 0:
        if c == 0:
            print("Уравнение 0 = 0, любое x является решением")
        else:
            print("Уравнение не имеет решений")
    else:
        t = -c / b
        if t < 0:
            print("Нет действительных корней")
        elif t == 0:
            print("x = 0")
        else:
            x1 = t ** 0.5
            x2 = -x1
            print("x1 =", x1)
            print("x2 =", x2)
else:
    D = b * b - 4 * a * c
    print("D =", D)

    if D < 0:
        print("Нет действительных корней")
    elif D == 0:
        t = -b / (2 * a)
        if t < 0:
            print("Нет действительных корней")
        elif t == 0:
            print("x = 0")
        else:
            x1 = t ** 0.5
            x2 = -x1
            print("x1 =", x1)
            print("x2 =", x2)
    else:
        t1 = (-b - D ** 0.5) / (2 * a)
        t2 = (-b + D ** 0.5) / (2 * a)

        if t1 > 0:
            x1 = t1 ** 0.5
            x2 = -x1
            print("x1 =", x1)
            print("x2 =", x2)
```

```
        elif t1 == 0:
            print("x = 0")

        if t2 > 0:
            x3 = t2 ** 0.5
            x4 = -x3
            print("x3 =", x3)
            print("x4 =", x4)
        elif t2 == 0:
            print("x = 0")

        if t1 < 0 and t2 < 0:
            print("Нет действительных корней")
```

Структурная парадигма:

Python:

```
def vvod(name):
    while True:
        try:
            return float(input("Введите " + name + ": "))
        except ValueError:
            print("Это не число, введите число")


def vvesti_koef():
    a = vvod("A")
    b = vvod("B")
    c = vvod("C")
    return a, b, c


def reshenie_a_0(b, c):
    if b == 0:
        if c == 0:
            print("Уравнение 0 = 0, любое x является решением")
        else:
            print("Уравнение не имеет решений")
    else:
        t = -c / b
        if t < 0:
            print("Нет действительных корней")
        elif t == 0:
            print("x = 0")
        else:
            print("x =", t ** 0.5)
            print("x =", -t ** 0.5)


def reshenie(a, b, c):
    D = b * b - 4 * a * c
    print("D =", D)
```

```
    if D < 0:
        print("Корней нет")
    elif D == 0:
        t = -b / (2 * a)
        if t < 0:
            print("Корней нет")
        elif t == 0:
            print("x = 0")
        else:
            print("x =", t ** 0.5)
            print("x =", -t ** 0.5)
    else:
        t1 = (-b + D ** 0.5) / (2 * a)
        t2 = (-b - D ** 0.5) / (2 * a)

        if t1 > 0:
            print("x =", t1 ** 0.5)
            print("x =", -t1 ** 0.5)
        elif t1 == 0:
            print("x = 0")

        if t2 > 0:
            print("x =", t2 ** 0.5)
            print("x =", -t2 ** 0.5)
        elif t2 == 0:
            print("x = 0")

        if t1 < 0 and t2 < 0:
            print("Корней нет")


def main():
    a, b, c = vvesti_koef()
    if a == 0:
        reshenie_a_0(b, c)
    else:
        reshenie(a, b, c)


if __name__ == "__main__":
    main()
```

Объектно-ориентированная парадигма:

C++:

```
#include <iostream>
#include <cmath>
using namespace std;

class U {
public:
    double a, b, c;
```

```
void vvod() {
    cout << "Введите A: ";
    cin >> a;
    cout << "Введите B: ";
    cin >> b;
    cout << "Введите C: ";
    cin >> c;
}

void reshenie() {
    if (a == 0) {
        reshenie_a_0();
        return;
    }

    double D = b * b - 4 * a * c;
    cout << "D = " << D << endl;

    if (D < 0) {
        cout << "Корней нет" << endl;
        return;
    }

    if (D == 0) {
        double t = -b / (2 * a);
        kor(t);
        return;
    }

    double t1 = (-b + sqrt(D)) / (2 * a);
    double t2 = (-b - sqrt(D)) / (2 * a);
    kor(t1);
    kor(t2);
}

void reshenie_a_0() {

    if (b == 0) {
        if (c == 0) {
            cout << "Уравнение 0 = 0, любое x является решением" << endl;
        } else {
            cout << "Уравнение не имеет решений" << endl;
        }
        return;
    }

    double t = -c / b;
    kor(t);
}
```

```
    void kor(double t) {
        if (t < 0) return;
        if (t == 0) {
            cout << "x = 0" << endl;
        } else {
            cout << "x = " << sqrt(t) << endl;
            cout << "x = " << -sqrt(t) << endl;
        }
    }
};

int main() {
    setlocale(LC_ALL, "ru_RU.UTF-8");
    U u;
    u.vvod();
    u.reshenie();
    return 0;
}
```

Результаты выполнения программы

```
PS C:\Users\User\Desktop\vs code\sem2> & "C:\Program Files\Python313\python.exe" "c:/Users/User/Desktop/vs code/sem2/sem 3 python/
lr1.py"
Введите A: & "C:\Program Files\Python313\python.exe" "c:/Users/User/Desktop/vs code/sem2/sem 3 python/lr1.py"
Это не число, введите число
Введите A: 1
Введите B: ыы
Это не число, введите число
Введите B: 2
Введите C: 3
D = -8.0
Нет действительных корней
```

```
lr1_1.py"
Введите A: pop
Это не число, введите число
Введите A: 1
Введите B: -2
Введите C: 1
D = 0.0
x = 1.0
x = -1.0
```

```
PS C:\Users\User\Desktop\vs code\sem2>  ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\User\.vscode\extensions\ms-vscode.cpptools-1.35.2-w
in32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-e0cvdxzg.dsh' '--stdout=Microsoft-MIEngine-Out
-mfakqjhs.1ul' '--stderr=Microsoft-MIEngine-Error-hfu3n4sy.zwk' '--pid=Microsoft-MIEngine-Pid-g011tbb2.2xe' '--dbgExe=C:\msys64\uc
rt64\bin\gdb.exe' '--interpreter=mi'
Введите A: 4
Введите B: -3
Введите C: 8
D = -119
Корней нет
```